No strongly regular graph srg$(99, 14, 1, 2)$ admits an automorphism of order seven

Blueprint of the formal proof

Utku Okur

July 23, 2026

Contents

Chapter 1

Strongly regular graphs and the statement

Definition 1 (Strongly regular graph)**.** A finite graph Γ on n vertices is *strongly regular with parameters* (n, k, λ, μ) if it is k-regular, any two adjacent vertices have exactly λ common neighbours, and any two distinct non-adjacent vertices have exactly μ common neighbours. Whether a graph with parameters $(99, 14, 1, 2)$ — the *Conway 99-graph* — exists is open; the present development constrains its possible symmetries.

Lean: `SimpleGraph.IsSRGWith`

source: `mathlib / Lean core` ✓

Definition 2 (Automorphism group)**.** An *isomorphism* between graphs Γ and Γ' is a bijection of vertex sets under which adjacency in Γ corresponds exactly to adjacency in Γ'. The isomorphisms from a graph Γ to itself form a group, the *automorphism group* $\operatorname{Aut}\Gamma$. For a finite graph this group is finite, and its order $|\operatorname{Aut}\Gamma|$ is the quantity constrained by the main theorem: the formal statements are about the cardinality of the type of self-isomorphisms $\Gamma \cong \Gamma$.

Lean: `SimpleGraph.Iso`

source: `mathlib / Lean core` ✓

Theorem 3 (Main theorem)**.** *Let Γ be a strongly regular graph with parameters $(99, 14, 1, 2)$, on any vertex set. Then $7 \nmid |\operatorname{Aut}\Gamma|$; in particular Γ has no automorphism of order 7.*

Lean: *`ConwayO7.no_aut_seven`*

source: *`ConwayO7/Final.lean`* ✓

uses: def:srg, st-aut, st-conditional, thm:encoding-complete, thm:coverage, fact:certificates

The proof is a chain of reductions. First, group theory reduces the statement to the non-existence of a graph invariant under one *fixed* permutation σ_0 of order 7 (§2). Second, such invariant graphs are encoded as Boolean valuations satisfying an explicit propositional formula Φ; the encoding is proved *complete*: if an invariant graph exists, Φ is satisfiable (§3, §4). Third, Φ is proved unsatisfiable by a partition of the valuation space into 98,536 cells, each refuted by a checked certificate (§5, §6). The interface between the first two steps and the last is isolated in the following definition and conditional theorem.

Definition 4 (Encoding invariant graphs)**.** Let F be a propositional formula in conjunctive normal form. Say that F *encodes the σ_0-invariant strongly regular graphs* if for every graph H on the vertex set $\{0, 1, \ldots, 98\}$ which is strongly regular with parameters $(99, 14, 1, 2)$ and satisfies $H(\sigma_0 u, \sigma_0 v) \Leftrightarrow H(u, v)$ for all vertices u, v, there exists a Boolean valuation satisfying F. Note the direction: only

"graph ⇒ satisfying valuation" is required. If F were satisfiable but meaningless the main theorem would become vacuous, not wrong; it is the *unsatisfiability* of F that does the refuting.

Lean: ConwayO7.EncodesInvariantSRG

source: ConwayO7/Pipeline.lean ✓

uses: def:srg

Theorem 5 (Conditional main theorem)**.** *Let F be an unsatisfiable formula which encodes the σ_0-invariant strongly regular graphs in the sense of Definition 4. Then no strongly regular graph with parameters $(99, 14, 1, 2)$, on any finite vertex set, has automorphism group of order divisible by 7.*

Lean: ConwayO7.no_aut_seven_of_unsat

source: ConwayO7/Pipeline.lean ✓

uses: def:srg, st-aut, st-encodes, thm:reduction

Proof. Suppose Γ were such a graph with $7 \mid |\operatorname{Aut}\Gamma|$. By the group-theoretic reduction (Theorem 25), there would then be a σ_0-invariant $\operatorname{srg}(99, 14, 1, 2)$ on $\{0, \dots, 98\}$; since F encodes these graphs, F would be satisfiable, contradicting the hypothesis. □

The remainder of the blueprint instantiates the two hypotheses for the concrete formula Φ of Definition 34: encoding completeness is Theorem 87, and unsatisfiability is delivered by Theorem 110 together with Fact 119.

Chapter 2

From an order-seven automorphism to a canonical one

This chapter carries out the group-theoretic reduction: the statement "$7 \nmid |\operatorname{Aut}\Gamma|$ for every $\mathrm{srg}(99, 14, 1, 2)$ Γ" is reduced to a single concrete assertion about one explicit permutation σ_0 of $\{0, \dots, 98\}$, namely that no σ_0-invariant $\mathrm{srg}(99, 14, 1, 2)$ exists (Theorem 25). The mathematical content follows Cesarz–Woldar: an order-seven automorphism of an $\mathrm{srg}(99, 14, 1, 2)$ fixes exactly one vertex, hence has cycle type $[1, 7^{14}]$, hence is conjugate in the symmetric group to the canonical block rotation σ_0.

2.1 Fixed points of prime-order permutations

We begin with elementary counting lemmas about permutations π with $\pi^p = 1$ for a prime p. Throughout, $\operatorname{Fix}\pi$ denotes the set of fixed points and $\operatorname{supp}\pi$ its complement, the support.

Lemma 6 (Support is a multiple of p)**.** *Let π be a permutation of a finite type with $\pi^p = 1$ for a prime p. Then $|\operatorname{supp}\pi|$ is a multiple of p.*

Lean: `ConwayO7.card_support_of_pow_prime_eq_one`

source: `ConwayO7/CycleType.lean` ✓

Proof. ✓Either $\pi = 1$, with empty support, or $\operatorname{ord}\pi = p$; in the latter case the cycle type of π is a multiset of copies of p, and the support is the disjoint union of the cycles. □

Lemma 7 (Fixed points and support partition the domain)**.** *For any permutation π of a finite type α, $|\operatorname{Fix}\pi| + |\operatorname{supp}\pi| = |\alpha|$.*

Lean: `ConwayO7.card_filter_fixed_add_card_support`

source: `ConwayO7/CycleType.lean` ✓

Lemma 8 (Fixed-point congruence)**.** *Let π be a permutation with $\pi^p = 1$, p prime, and let s be a finite π-invariant set. Then*

$$|\{a \in s \mid \pi a = a\}| \equiv |s| \pmod{p}.$$

Lean: `ConwayO7.card_filter_fixed_modEq`

source: `ConwayO7/CycleType.lean` ✓

Proof. ✓
uses: lem:grp-support-multiple, lem:grp-fix-support Since π is injective and s is finite, $\pi(s) \subseteq s$ upgrades to $\pi(s) = s$, so π restricts to a permutation π' of s with $(\pi')^p = 1$. By Lemmas 6 and 7 applied to π', the fixed points of π inside s number $|s| - pm$ for some m. (We will apply this with $p = 7$ to the vertex set and to a neighbourhood, and with $p = 2$ to a matching involution.) □

Proposition 9 (Cycle type from a fixed-point count)**.** *Let $\pi \neq 1$ be a permutation of a finite type of cardinality $f + pm$ with $\pi^p = 1$, p prime, and exactly f fixed points. Then the cycle type of π is $[p^m]$.*
Lean: `ConwayO7.cycleType_eq_replicate_of_fixed`
source: `ConwayO7/Canonical.lean` ✓

Proof. ✓
uses: lem:grp-fix-support Since $\pi \neq 1$ and $\pi^p = 1$, the order of π is exactly p, so its cycle type is $[p^n]$ for some $n \geq 1$. By Lemma 7 the support has pm elements, and the cycle lengths sum to the support size, so $pn = pm$ and $n = m$. □

2.2 Local structure of an $\mathrm{srg}(99, 14, 1, 2)$

Fix an $\mathrm{srg}(99, 14, 1, 2)$ Γ on a vertex set V, and write $N(x)$ for the neighbourhood of x and $N(x, y) = N(x) \cap N(y)$ for a common neighbourhood.

Lemma 10 ($\lambda = 1$: unique triangles)**.** *If x and u are adjacent then $N(x, u)$ is a singleton: every edge of Γ lies in exactly one triangle.*
Lean: `ConwayO7.commonNeighbors_eq_singleton_of_adj`
source: `ConwayO7/CycleType.lean` ✓
uses: def:srg

Lemma 11 ($\mu = 2$: common-neighbour pairs)**.** *If $x \neq w$ are nonadjacent then $N(x, w) = \{a, b\}$ for two distinct vertices $a \neq b$.*
Lean: `ConwayO7.commonNeighbors_eq_pair_of_not_adj`
source: `ConwayO7/CycleType.lean` ✓
uses: def:srg

Lemma 12 (Injectivity of the Γ_2-labelling)**.** *Let $w, w' \neq x$ be nonadjacent to x. If $N(x, w) = N(x, w')$ then $w = w'$: a vertex at distance two from x is determined by its pair of common neighbours with x.*
Lean: `ConwayO7.eq_of_commonNeighbors_eq`
source: `ConwayO7/CycleType.lean` ✓
uses: def:srg

Proof. ✓
uses: lem:grp-unique-triangle, lem:grp-mu-pair Write $N(x, w) = \{a, b\}$ with $a \neq b$. The labels a, b are nonadjacent: an edge ab would put the two distinct vertices x and w into the unique triangle over ab granted by $\lambda = 1$. Hence by $\mu = 2$ the pair $\{a, b\}$ has exactly two common neighbours; both $\{x, w\}$ and $\{x, w'\}$ are contained in $N(a, b)$ and have the right size, so $\{x, w\} = N(a, b) = \{x, w'\}$ and $w = w'$. □

Lemma 13 (Equivariance of common neighbourhoods)**.** *Let π be a permutation of V with $\Gamma(\pi u, \pi v) \Leftrightarrow \Gamma(u, v)$ for all u, v. Then $\pi(N(v, w)) \subseteq N(\pi v, \pi w)$.*

Lean: `ConwayO7.mapsTo_commonNeighbors`

source: `ConwayO7/CycleType.lean` ✓

Proposition 14 (Rigidity)**.** *An adjacency-preserving permutation of an* $\mathrm{srg}(99, 14, 1, 2)$ *that fixes a vertex x and every neighbour of x is the identity.*

Lean: `ConwayO7.eq_one_of_fixes_neighbors`

source: `ConwayO7/CycleType.lean` ✓

uses: def:srg

Proof. ✓

uses: lem:grp-label-injective, lem:grp-equivariance Let π fix x and $N(x)$ pointwise, and let $v \notin \{x\} \cup N(x)$. The common neighbourhood $N(x, v)$ consists of neighbours of x, hence is pointwise fixed; by equivariance (applied to π and to π^{-1}) it follows that $N(x, \pi v) = N(x, v)$. Since πv is again nonadjacent to x and distinct from x, injectivity of the labelling (Lemma 12) forces $\pi v = v$. □

2.3 The fixed-point count and the cycle type

Theorem 15 (Exactly one fixed vertex)**.** *Let Γ be an* $\mathrm{srg}(99, 14, 1, 2)$ *and let π be an adjacency-preserving permutation of its vertices with* $\operatorname{ord}\pi = 7$*. Then π fixes exactly one vertex.*

Lean: `ConwayO7.card_filter_fixed_eq_one`

source: `ConwayO7/CycleType.lean` ✓

uses: def:srg

Proof sketch. ✓

uses: lem:grp-fixed-congruence, lem:grp-unique-triangle, lem:grp-mu-pair, lem:grp-equivariance, lem:grp-rigidity *Step 0.* By the congruence (Lemma 8, $p = 7$, $s = V$), $|\operatorname{Fix}\pi| \equiv 99 \equiv 1 \pmod 7$; in particular a fixed vertex x exists.

Step 1. Let $s = N(x) \cap \operatorname{Fix}\pi$ be the set of fixed neighbours of x. By $\lambda = 1$, the map sending $u \in N(x)$ to the apex of the unique triangle over the edge xu is a fixed-point-free involution of $N(x)$; by equivariance and uniqueness of the apex it preserves s, so (Lemma 8 with $p = 2$) $|s|$ is even. Applying the congruence with $p = 7$ to the invariant set $N(x)$ gives $|s| \equiv 14 \equiv 0 \pmod 7$; together with $|s| \le 14$ this leaves $|s| \in \{0, 14\}$.

Step 2. If $|s| = 14$ then π fixes x and all of $N(x)$, so $\pi = 1$ by rigidity (Proposition 14), contradicting $\operatorname{ord}\pi = 7$. Hence $s = \emptyset$.

Step 3. Suppose $w \neq x$ were a second fixed vertex. It cannot be adjacent to x (it would lie in s). Otherwise π permutes the two common neighbours $\{a, b\} = N(x, w)$; neither is fixed (both are neighbours of x), so π swaps them. Then π^2 fixes a, and $\pi = (\pi^2)^4$ fixes $a \in N(x)$ — contradiction. So $\operatorname{Fix}\pi = \{x\}$. □

Corollary 16 (Cycle type of an order-seven automorphism)**.** *Let Γ be an* $\mathrm{srg}(99, 14, 1, 2)$ *and $\sigma \in \operatorname{Aut}\Gamma$ with* $\operatorname{ord}\sigma = 7$*. Then the underlying vertex permutation of σ has cycle type* $[7^{14}]$.

Lean: `ConwayO7.cycleType_autToPerm`

source: `ConwayO7/CycleType.lean` ✓

uses: def:srg, def:grp-autToPerm, thm:grp-cycle-type

Proof. ✓

uses: lem:grp-autToPerm-injective An injective homomorphism preserves orders, so the underlying permutation also has order 7 and preserves adjacency; apply Theorem **??**. □

2.4 The canonical permutation σ_0

Definition 17 (The canonical permutation)**.** On the vertex set $\{0, \dots, 98\}$, partition the nonzero vertices into fourteen consecutive blocks $B_i = \{1 + 7i, \dots, 7 + 7i\}$, $0 \le i \le 13$. Let σ_0 be the permutation fixing 0 and rotating each block cyclically:

$$\sigma_0(0) = 0, \qquad \sigma_0(1 + 7i + j) = 1 + 7i + ((j + 1) \bmod 7) \quad (0 \le j < 7).$$

Formally, σ_0 is assembled from the explicit forward-rotation formula and its backward-rotation companion (positions advance by 1, respectively by $6 \equiv -1$, modulo 7).

Lean: `ConwayO7.sigma0, ConwayO7.sigma0Fun, ConwayO7.sigma0InvFun, ConwayO7.sigma0_val`

source: `ConwayO7/Canonical.lean` ✓

uses: lem:grp-sigma0-inverse

Proof. ✓On a nonzero vertex $1+7i+j$ the composite advances the position by $1+6 \equiv 0 \pmod 7$ and leaves the block index i unchanged; the vertex 0 is fixed by both maps. □

Lemma 18 (Blocks are constant, positions advance)**.** *For a nonzero vertex $v = 1 + 7i + j$ and every $k \ge 0$,*

$$\sigma_0^k(v) = 1 + 7i + ((j + k) \bmod 7) :$$

iterating σ_0 never leaves the block of v and advances the position within the block by k modulo 7.

Lean: `ConwayO7.sigma0_iterate_val`

source: `ConwayO7/Canonical.lean` ✓

uses: def:sigma0

Proof. ✓Induction on k: a nonzero vertex stays nonzero (its position formula keeps it in the block B_i), so the defining formula of σ_0 applies at every step and adds 1 to the position, modulo 7. □

Lemma 19 (σ_0 has order dividing seven)**.** $\sigma_0^7 = 1$.

Lean: `ConwayO7.sigma0_pow_seven, ConwayO7.sigma0_iterate_seven`

source: `ConwayO7/Canonical.lean` ✓

uses: def:sigma0

Proof. ✓

uses: lem:grp-sigma0-iterate The vertex 0 is fixed; on a nonzero vertex, Lemma 18 with $k = 7$ advances the position by $7 \equiv 0 \pmod 7$. □

Lemma 20 (Fixed points of σ_0)**.** $\sigma_0(v) = v$ *if and only if* $v = 0$.

Lean: `ConwayO7.sigma0_fixed_iff`

source: `ConwayO7/Canonical.lean` ✓

uses: def:sigma0

Proof. ✓On a nonzero vertex the position advances, and $(j + 1) \bmod 7 = j$ is impossible for $0 \le j < 7$. □

Lemma 21 (Fixed-point count of σ_0)**.** σ_0 *has exactly one fixed point; in particular* $\sigma_0 \ne 1$.

Lean: `ConwayO7.sigma0_fixed_card, ConwayO7.sigma0_ne_one`

source: `ConwayO7/Canonical.lean` ✓

uses: lem:grp-sigma0-fixed-iff

Lemma 22 (Cycle type of σ_0)**.** *The cycle type of σ_0 is $[7^{14}]$ — the same as that of any order-seven automorphism of an* $\mathrm{srg}(99,14,1,2)$ *(Corollary 16).*

Lean: `ConwayO7.sigma0_cycleType`

source: `ConwayO7/Canonical.lean` ✓

uses: def:sigma0

Proof. ✓

uses: lem:grp-cycletype-of-fixed, lem:grp-sigma0-order, lem:grp-sigma0-fixed-card Apply Proposition 9 with $p = 7$, $f = 1$, $m = 14$ and $99 = 1 + 7 \cdot 14$, using $\sigma_0^7 = 1$, $\sigma_0 \neq 1$ and the fixed-point count. □

2.5 Transport and the reduction

Lemma 23 (Transport of strong regularity)**.** *If $G \simeq_g H$ is an isomorphism of finite graphs and G is an* $\mathrm{srg}(n,k,\lambda,\mu)$*, then so is H.*

Lean: `ConwayO7.isSRGWith_of_iso`

source: `ConwayO7/Canonical.lean` ✓

uses: def:srg

Proof. ✓The isomorphism identifies neighbourhoods and common neighbourhoods vertex-by-vertex, so all four parameters are preserved. □

Theorem 24 (WLOG σ_0)**.** *If some* $\mathrm{srg}(99,14,1,2)$ Γ*, on any vertex set, admits an automorphism of order 7, then some* $\mathrm{srg}(99,14,1,2)$ *on the vertex set $\{0,\dots,98\}$ is invariant under σ_0.*

Lean: `ConwayO7.exists_sigma0_invariant`

source: `ConwayO7/Canonical.lean` ✓

uses: def:srg, def:sigma0

Proof sketch. ✓

uses: lem:grp-srg-transport, cor:grp-aut-cycle-type, lem:grp-sigma0-cycle-type Transport Γ to $\{0,\dots,98\}$ along any bijection of vertex sets (possible since $|V| = 99$); by Lemma 23 the transported graph is again an $\mathrm{srg}(99,14,1,2)$, and the automorphism pushes through to an order-seven adjacency-preserving permutation π_1. By Corollary 16 and Lemma 22, π_1 and σ_0 have the same cycle type $[7^{14}]$, hence are conjugate in the symmetric group (mathlib's `Equiv.Perm.isConj_iff_cycleType_eq`); say $\tau\pi_1\tau^{-1} = \sigma_0$. Relabelling once more by τ yields an $\mathrm{srg}(99,14,1,2)$ invariant under σ_0 itself. □

Theorem 25 (Reduction to σ_0)**.** *Suppose no* $\mathrm{srg}(99,14,1,2)$ *on the vertex set $\{0,\dots,98\}$ is invariant under σ_0. Then $7 \nmid |\mathrm{Aut}\,\Gamma|$ for every* $\mathrm{srg}(99,14,1,2)$ Γ.

Lean: `ConwayO7.seven_not_dvd_card_aut_of_no_sigma0_invariant`

source: `ConwayO7/Canonical.lean` ✓

uses: def:srg, def:sigma0

Proof. ✓

uses: lem:cauchy, thm:grp-wlog If $7 \mid |\mathrm{Aut}\,\Gamma|$ then by the Cauchy step (Lemma **??**) some automorphism has order 7, and Theorem 24 produces a σ_0-invariant $\mathrm{srg}(99,14,1,2)$ on $\{0,\dots,98\}$ — contradicting the hypothesis. The hypothesis is exactly the statement encoded as a CNF formula in Chapter 3 and refuted by the verified proof checker, completing this branch of Theorem 3. □

Chapter 3

The orbit quotient and the propositional encoding

By Theorem 25 it suffices to decide whether some graph invariant under the canonical permutation σ_0 is strongly regular with parameters $(99, 14, 1, 2)$. This chapter carries out the reduction of that question to a single propositional formula Φ, and proves the direction that the certificate method needs: every invariant srg would give rise to a satisfying valuation of (the pre-normalisation part of) Φ. The chapter has five strands: the orbit quotient of the edge set (§3.1), the formula itself as a mathematical object, a Hoare-style framework for reasoning about clause-emitting transformations, the soundness of the three counting gadgets, and finally the bridge from strong regularity to the semantic counting invariants.

3.1 The orbit quotient

Definition 26 (Pair orbits and representatives)**.** The cyclic group $\langle\sigma_0\rangle$ acts on unordered pairs of vertices $\{u, v\} \subseteq \{0, \dots, 98\}$, $u \neq v$, by $\sigma_0 \cdot \{u, v\} = \{\sigma_0(u), \sigma_0(v)\}$. Write $\mathrm{orb}(u, v)$ for the index of the orbit of $\{u, v\}$, orbits being numbered $0, 1, 2, \dots$ in order of first appearance as the pairs are enumerated lexicographically, and for each orbit index o let $\mathrm{rep}(o)$ denote the first pair of orbit o in that enumeration (its *representative*). The map orb is symmetric in its two arguments and constant along the action: $\mathrm{orb}(\sigma_0 u, \sigma_0 v) = \mathrm{orb}(u, v)$.

Lean: `ConwayO7.orbitOf`, `ConwayO7.orbitRep`, `ConwayO7.orbitOf_swap`, `ConwayO7.orbitOf_sigma0`

source: `ConwayO7/Orbits.lean` ✓

uses: def:sigma0

Lemma 27 (Orbit structure of pairs)**.** *The action of* $\langle\sigma_0\rangle \cong \mathbb{Z}/7$ *on the* $\binom{99}{2} = 4851$ *unordered pairs of vertices has exactly* 693 *orbits, each of size* 7*; equivalently,* orb *takes exactly the values* $0, \dots, 692$*, and every pair is reached from its orbit's representative by iterating the action.*

Lean: `ConwayO7.numOrbits_eq`, `ConwayO7.orbitOf_lt`, `ConwayO7.orbitRep_reaches`

source: `ConwayO7/Orbits.lean` ✓

uses: def:enc-orbitmap

Proof. ✓No pair is fixed by σ_0 (its two 7-cycles share no point with any pair twice), so every orbit has size exactly 7 and $4851/7 = 693$. In the formalisation the orbit table is computed once and the three statements are verified by evaluation. □

Definition 28 (The graph of an orbit valuation)**.** For $\omega\colon \mathbb{N} \to \{0,1\}$ let G_ω be the graph on $\{0,\dots,98\}$ in which distinct u, v are adjacent exactly when $\omega(\mathrm{orb}(u,v)) = 1$. Only the values $\omega(0),\dots,\omega(692)$ matter, so G_ω is determined by a word in $\{0,1\}^{693}$.

Lean: `ConwayO7.graphOfOrbits`

source: `ConwayO7/Orbits.lean` ✓

uses: def:enc-orbitmap, lem:orbits

Lemma 29 (Invariance of the induced graph)**.** *For every ω the graph G_ω is σ_0-invariant: $\sigma_0(u) \sim \sigma_0(v)$ in G_ω if and only if $u \sim v$.*

Lean: `ConwayO7.graphOfOrbits_invariant`

source: `ConwayO7/Orbits.lean` ✓

uses: def:enc-graphoforbits, def:sigma0

Proof. ✓Immediate from the constancy of orb along the action (Definition 26) and injectivity of σ_0. □

Lemma 30 (Faithfulness of the orbit reduction)**.** *Let H be a σ_0-invariant graph on $\{0,\dots,98\}$ and define $\omega_H(o) = 1$ exactly when the representative pair $\mathrm{rep}(o)$ is an edge of H. Then $G_{\omega_H} = H$: an invariant graph is reconstructed exactly from its 693 orbit bits.*

Lean: `ConwayO7.orbitFunOf, ConwayO7.graphOfOrbits_orbitFunOf`

source: `ConwayO7/Orbits.lean` ✓

uses: def:enc-graphoforbits, lem:orbits

Proof. ✓Given distinct u, v, the pair $\{u, v\}$ is reached from $\mathrm{rep}(\mathrm{orb}(u,v))$ by iterating the action (Lemma 27); each step preserves adjacency in H by invariance, so $u \sim_H v$ holds if and only if the representative pair is an edge, which is the definition of $\omega_H(\mathrm{orb}(u,v))$. □

Proposition 31 (Invariant graphs are orbit functions)**.** *A graph H on $\{0,\dots,98\}$ is σ_0-invariant if and only if $H = G_\omega$ for some $\omega\colon \mathbb{N} \to \{0,1\}$. This is the verified reduction from 4851 edge variables to 693 orbit variables.*

Lean: `ConwayO7.invariant_iff_exists_orbitFun`

source: `ConwayO7/Orbits.lean` ✓

uses: lem:enc-invariance, lem:enc-faithful

Proof. ✓Combine Lemma 29 (backward direction) with Lemma 30 (forward direction, witness ω_H). □

3.2 The formula

Definition 32 (Emission states)**.** An *emission state* is a pair $s = (t, C)$ of a threshold $t \in \mathbb{N}$ — the highest variable index allocated so far — and a finite sequence C of clauses, each clause a finite list of nonzero integer literals. A valuation $a\colon \mathbb{N} \to \{0,1\}$ *satisfies* s if it satisfies every clause of C (a positive literal v is true when $a(v) = 1$, a negative literal $-v$ when $a(v) = 0$). The state is *well-formed* if $t \geq 693$ and every literal of every clause mentions a variable $\leq t$. The generator below starts from the initial state $(693, \varnothing)$, whose first 693 variables $x_1,\dots,x_{693}$ are the orbit variables, and only ever appends clauses and raises the threshold.

Lean: `ConwayO7.Encoder.St, ConwayO7.Encoder.SatAll, ConwayO7.Encoder.WF`

source: `ConwayO7/Encoder.lean` ✓

Definition 33 (Orbit literals and orbit valuations)**.** For distinct vertices u, v the *orbit literal* of the pair is the (positive) literal of variable $\mathrm{orb}(u, v) + 1$; thus the adjacency of u, v in G_ω is expressed by one of the variables $x_1, \ldots, x_{693}$. Conversely each $\omega \colon \mathbb{N} \to \{0, 1\}$ induces the valuation $a_\omega(v) = \omega(v - 1)$, under which the orbit literal of (u, v) is true exactly when $u \sim v$ in G_ω.

Lean: `Conway07.Encoder.evarI, Conway07.Encoder.assignOfOrbits, Conway07.Encoder.litSat_evarI`

source: `Conway07/Encoder.lean` ✓

uses: def:enc-orbitmap, def:enc-graphoforbits, def:enc-state

Definition 34 (The formula Φ)**.** Φ is a propositional formula in conjunctive normal form over the orbit variables $x_1, \ldots, x_{693}$ together with auxiliary counter variables, obtained by running four clause-emitting sections from the initial state and reading off the final clause sequence. Its clauses express, for the graph G_ω associated to a valuation ω of $x_1, \ldots, x_{693}$:

1. *regularity*: every vertex has exactly 14 neighbours;
2. *common-neighbour counts*: for each orbit representative pair (u, v), the number of common neighbours of u, v equals 2 minus the adjacency bit of (u, v) (i.e. $\lambda = 1$, $\mu = 2$);
3. *normalisation*: the neighbourhood of the fixed vertex 0 is the union of the first two 7-blocks;
4. *minimality*: the word $\omega \in \{0, 1\}^{693}$ is lexicographically minimal in its orbit under the 101 relabelling generators of Definition 60.

The cardinality conditions are rendered as clauses through standard counting networks (a sequential counter and a modulo-9 totalizer, §3.4); the minimality conditions through chained lexicographic comparisons. Φ has 432,882 variables and 1,131,708 clauses.

Lean: `Conway07.Encoder.mk07St, Conway07.Encoder.mk07Cnf`

source: `Conway07/Encoder.lean` ✓

uses: def:enc-state, def:enc-evar, lem:orbits, def:relabel, lem:sym1

Lemma 35 (No degenerate literals)**.** *No clause of Φ contains the literal 0; every literal denotes a genuine variable with a polarity.*

Lean: `Conway07.Encoder.mk07_no_zero`

source: `Conway07/SatFrame.lean` ✓

uses: def:phi

Proof. ✓Checked by evaluating the generator and scanning all emitted literals. □

Lemma 36 (Satisfaction transfers to the formula)**.** *A valuation satisfies the final emission state of the generator (in the integer-literal sense of Definition 32) if and only if the corresponding 0-based valuation satisfies Φ as a formal CNF. In particular, if some valuation satisfies every generated clause then Φ is satisfiable.*

Lean: `Conway07.Encoder.satAll_iff_cnfSat, Conway07.Encoder.sat_mk07Cnf_of_satAll`

source: `Conway07/SatFrame.lean` ✓

uses: def:phi, lem:enc-nozero

Proof. ✓The two satisfaction relations agree literal by literal once no literal is 0 (Lemma 35); the translation is the shift by one between 1-based and 0-based variable indexing. □

Computed Fact 37 (Byte identity)**.** The formula generated inside Lean equals the parsed content of the distributed file `o7.cnf` (SHA-256 pinned), and re-serialising that content reproduces the file byte for byte, header included. All certificates of later chapters therefore speak about exactly the formula Φ of Definition 34.

Lean: `ConwayO7.mkO7Cnf_eq, ConwayO7.o7cnf_bytes`

source: `ConwayO7/Data/EncoderMatch.lean` ✓

uses: def:phi

3.3 The verification framework

The generator is a composition of dozens of clause-emitting transformations. Their correctness is captured by a single mathematical notion.

Definition 38 (Emission triples)**.** Let P and Q be conditions on valuations and let f be a transformation of emission states. The *emission triple* $\{P\}\, f\, \{Q\}$ holds if f never lowers the threshold and, for every well-formed state s with threshold t, every valuation a satisfying s and the precondition P, there exists a valuation a' such that:

1. $a'(v) = a(v)$ for all $v \le t$ (only *fresh* auxiliary variables, above the threshold, are reassigned);
2. a' satisfies every clause of $f(s)$ — the old clauses and the newly emitted ones;
3. $f(s)$ is again well-formed, and $Q(a')$ holds.

Thus a triple asserts: whatever clauses have been emitted so far, the new gadget's clauses can always be satisfied by choosing values for its own fresh auxiliary variables, provided the semantic condition P holds on the protected variables.

Lean: `ConwayO7.Encoder.GTrip`

source: `ConwayO7/SatFrame.lean` ✓

uses: def:enc-state

Lemma 39 (Composition of emission triples)**.** *Emission triples compose: from* $\{P\}\, f\, \{Q\}$ *and* $\{Q\}\, g\, \{R\}$ *follows* $\{P\}\, g \circ f\, \{R\}$*. More generally, if a condition* Inv *is an invariant of every step of a finite iteration, it is an invariant of the whole iteration.*

Lean: `ConwayO7.Encoder.GTrip.comp, ConwayO7.Encoder.GTrip.foldl_inv`

source: `ConwayO7/SatFrame.lean` ✓

uses: def:enc-gtrip

Proof. ✓Chain the two witnesses; the agreement below the original threshold persists because f only raises it. The iteration statement is induction on the list of steps. □

The three sections preceding the minimality constraints have the following semantic preconditions, stated purely as counting conditions on a valuation.

Definition 40 (Degree invariant)**.** A valuation a satisfies the *degree invariant* if for every vertex u exactly 14 of the 98 orbit literals $\{\, \mathrm{orb}(u, v) + 1 : v \neq u \,\}$ are true under a; i.e. every vertex of the graph read off the orbit variables has degree 14.

Lean: `ConwayO7.Encoder.DegInv`

source: `ConwayO7/DegSpec.lean` ✓

uses: def:enc-evar

Definition 41 (Common-neighbour invariant)**.** For a representative pair $r = (r_1, r_2)$ and a valuation a, let $c_r(a)$ be the number of vertices $w \notin \{r_1, r_2\}$ whose orbit literals with both r_1 and r_2 are true, *plus* 1 if the orbit literal of (r_1, r_2) itself is true. A valuation satisfies the *common-neighbour invariant* if $c_{\mathrm{rep}(o)}(a) = 2$ for every orbit $o < 693$. (Since $\mu - \lambda = 1$ for the parameters $(99, 14, 1, 2)$, the adjacent case $\lambda = 1$ and the non-adjacent case $\mu = 2$ are captured by the single value 2.)

Lean: `ConwayO7.Encoder.cnCountR, ConwayO7.Encoder.CnInv`

source: `ConwayO7/CnSpec.lean` ✓

uses: def:enc-evar, def:enc-orbitmap

Definition 42 (Level-1 symmetry condition)**.** A valuation a satisfies the *level-1 symmetry condition* if for each $i < 14$ the orbit variable of the pair $(0, 1 + 7i)$ has value 1 exactly when $i < 2$: the fixed vertex is adjacent to precisely the first two 7-cycles. This is the semantic content of the normalisation of Lemma 68.

Lean: `ConwayO7.Encoder.Sym1Holds`

source: `ConwayO7/SatFrame.lean` ✓

uses: def:enc-evar, lem:sym1

Definition 43 (The pre-lex invariant)**.** The *pre-lex invariant* is the conjunction of the degree invariant, the common-neighbour invariant, and the level-1 symmetry condition.

Lean: `ConwayO7.Encoder.PreLexInv`

source: `ConwayO7/SectionsCompose.lean` ✓

uses: def:enc-deginv, def:enc-cninv, def:enc-sym1holds

3.4 Soundness of the counting networks

The cardinality constraints of Φ are built from three networks. Their soundness statements all have the same shape: *whenever the semantic count is within the stated bound, the emitted clauses admit an extension of the valuation assigning only fresh auxiliary variables.*

Definition 44 (Unary representation of a count)**.** A vector of literals $(y_1, \ldots, y_m)$ *carries the unary representation* of a number k under a valuation a if y_i is true exactly when $i \le k$; when k is the number of true literals among a vector of inputs, we say the register carries the unary count of those inputs.

Lean: `ConwayO7.TotSpec.UnaryOut, ConwayO7.MtoSpec.UnaryOf`

source: `ConwayO7/TotSpec.lean` ✓

uses: def:enc-state

Proposition 45 (Soundness of the sequential counter)**.** *Fix input literals $\ell_1, \ldots, \ell_n$ over variables $\le T$ and a threshold $1 \le \tau$ with $\tau + 2 \le n$. For any valuation a under which at most τ of the inputs are true, the valuation extending a that assigns to each fresh register variable $s_{k,j}$ the truth value of "more than k of the first $k + j + 1$ inputs are true" satisfies every clause of the sequential at-most-τ counter, and agrees with a on all variables $\le T$.*

Lean: `ConwayO7.SeqSpec.spec_sat`

source: `ConwayO7/SeqSpec.lean` ✓

uses: def:enc-state

Proof. ✓Each clause relates a register to its predecessor and one input, and states one instance of the recurrence for the running count $m \mapsto \#\{i \le m : a(\ell_i) = 1\}$; the chosen witness makes every such instance a true arithmetic fact, the final clauses being exactly the bound "the count never exceeds τ". □

Lemma 46 (The totalizer merge step)**.** *Suppose, under a valuation a, two child registers carry the unary counts of their respective input sets and the parent register carries the unary count of the union. Then all merge clauses of the internal node — which say that i true literals on one side and j on the other force at least $i + j$ in the parent, and conversely — are satisfied by a itself; no fresh variables are involved in this step.*

Lean: `ConwayO7.TotSpec.toUA_sat`

source: `ConwayO7/TotSpec.lean` ✓

uses: def:enc-unary

Proposition 47 (Soundness of the totalizer)**.** *For any well-formed state s with threshold t, valuation a satisfying s, and input literals over variables $\leq t$: there is a valuation a' agreeing with a on all variables $\leq t$ that satisfies every clause of the totalizer built on those inputs, and under which the output register carries the unary count of the inputs.*

Lean: `ConwayO7.TotSpec.toTO_sat`

source: `ConwayO7/TotSpec.lean` ✓

uses: def:enc-unary, lem:enc-totmerge

Proof. ✓Structural induction on the balanced binary tree over the inputs. At each internal node the fresh output register is assigned the unary representation of the number of true leaves below it; the count at a node is the sum of the counts at its children, so Lemma 46 discharges the merge clauses, and the induction hypothesis handles the subtrees. □

Proposition 48 (Soundness of the modulo-9 totalizer)**.** *Let C be the number of true inputs under a (at least 9 inputs, over variables $\leq t$). There is an extension a' of a by fresh variables satisfying all clauses of the modulo-9 totalizer, under which the two output registers carry the unary representations of the quotient $\lfloor C/9 \rfloor$ and the remainder $C \bmod 9$ respectively.*

Lean: `ConwayO7.MtoSpec.mtoMTO_sat`

source: `ConwayO7/MtoSpec.lean` ✓

uses: def:enc-unary, prop:enc-totalizer

Proof. ✓Again induction over a binary tree, but each internal node now merges two quotient/remainder pairs: the remainders are added as unary numbers, a carry bit records whether their sum reached 9, and the quotients are added together with the carry. The witness assigns every fresh register its exact semantic value, so each merge clause becomes an instance of the identity $C = 9\lfloor C/9 \rfloor + (C \bmod 9)$ for the subtree counts. □

Proposition 49 (The at-most-k constraint via the modulo totalizer)**.** *Fix 98 input literals over variables $\leq t$ and $0 < k < 98$. If at most k of the inputs are true under a, then the modulo-9 totalizer together with its comparator clauses — which forbid every quotient/remainder combination exceeding k — is satisfied by an extension of a assigning only fresh variables.*

Lean: `ConwayO7.MtoSpec.mtoAtMost_sat`

source: `ConwayO7/MtoSpec.lean` ✓

uses: prop:enc-mto

Proof. ✓Take the extension of Proposition 48. A comparator clause rules out the joint configuration "quotient $\geq q$ and remainder $\geq r$" for each pair with $9q + r > k$; since the registers carry the true quotient and remainder of a count $C \leq k$, no forbidden configuration occurs. □

Lemma 50 (The degree gadget)**.** *Fix a vertex u and a valuation a under which exactly 14 of the 98 orbit literals at u are true. Then the exactly-14 gadget for u — an at-least-14 network run on*

the negated literals (as at-most-84) followed by an at-most-14 network, both modulo-9 totalizers — admits an extension of a by fresh variables satisfying all its clauses.

Lean: `ConwayO7.Encoder.encEqualsDeg_sat`

source: `ConwayO7/DegSpec.lean` ✓

uses: prop:enc-mto-atmost, def:enc-deginv, def:enc-evar

Proof. ✓Exactly 14 of 98 literals true means exactly 84 of the negated literals are true; apply Proposition 49 twice in sequence, composing the two fresh extensions. □

Proposition 51 (The degree section)**.** *The degree invariant is a valid pre- and postcondition for the degree gadget of each vertex, and hence for the entire degree section — the iteration of the gadget over all* 99 *vertices — as one emission triple.*

Lean: `ConwayO7.Encoder.degreeStep_trip, ConwayO7.Encoder.deg_section_trip`

source: `ConwayO7/DegSpec.lean` ✓

uses: def:enc-gtrip, lem:enc-gtrip-comp, lem:enc-deg-gadget

Proof. ✓The degree invariant concerns only the orbit variables (≤ 693), which every gadget's extension leaves untouched; so it survives each step, and Lemma 50 supplies each step's satisfiability. Compose by Lemma 39. □

Lemma 52 (The common-neighbour products)**.** *Fix a representative pair $r = (r_1, r_2)$. For each candidate common neighbour w the gadget allocates a fresh variable p_w with three clauses expressing $p_w \leftrightarrow (e_{r_1 w} \wedge e_{r_2 w})$ in terms of the orbit literals. Assigning each p_w its semantic value — the conjunction of the two adjacencies — satisfies all product clauses, and the recorded list of product literals evaluates to exactly the indicator sequence of common neighbourhood.*

Lean: `ConwayO7.Encoder.prodFold_sat`

source: `ConwayO7/CnSpec.lean` ✓

uses: def:enc-evar, def:enc-cninv

Lemma 53 (The exactly-2 gadget)**.** *Fix* 98 *literals over variables $\leq t$ of which exactly two are true under a. Then the exactly-2 gadget — a sequential at-most-96 counter on the negated literals (enforcing at least 2) followed by a sequential at-most-2 counter — admits an extension of a by fresh variables satisfying all its clauses.*

Lean: `ConwayO7.Encoder.specSeqEquals_sat`

source: `ConwayO7/CnSpec.lean` ✓

uses: prop:enc-seqcounter

Proof. ✓Exactly 2 true literals means exactly 96 true negations; apply Proposition 45 to each counter in turn and compose the extensions. □

Proposition 54 (The common-neighbour section)**.** *The common-neighbour invariant is a valid pre- and postcondition for the gadget of each orbit — the product stage followed by the exactly-2 gadget on the* 97 *product literals together with the pair's own orbit literal — and hence for the whole section over all* 693 *orbits, as one emission triple.*

Lean: `ConwayO7.Encoder.specCnStep_trip, ConwayO7.Encoder.cn_section_trip`

source: `ConwayO7/CnSpec.lean` ✓

uses: def:enc-gtrip, lem:enc-gtrip-comp, lem:enc-cn-product, lem:enc-cn-equals, def:enc-cninv

Proof. ✓By Lemma 52 the product literals plus the adjacency literal have exactly $c_r(a)$ true entries, which the invariant fixes at 2; Lemma 53 then extends the valuation over the two counters. The invariant mentions only orbit variables and survives every fresh extension; compose by Lemma 39. □

Lemma 55 (The normalisation section)**.** *The level-1 symmetry condition is a valid pre- and postcondition for the section emitting the* 14 *unit clauses that pin the neighbourhood of the fixed vertex: under the condition, each unit clause is already true and no fresh variables are needed.*

Lean: `ConwayO7.Encoder.sym1_section_trip`

source: `ConwayO7/SatFrame.lean` ✓

uses: def:enc-gtrip, lem:enc-gtrip-comp, def:enc-sym1holds

Proposition 56 (Pre-lex satisfiability)**.** *For every valuation of the orbit variables satisfying the pre-lex invariant (Definition 43) — that is, the arithmetic conditions (1)–(3) of Definition 34 — there is an extension assigning only auxiliary variables above* 693 *that satisfies every clause emitted by the degree, common-neighbour, and normalisation sections of the generator.*

Lean: `ConwayO7.Encoder.preLexSpec_trip`, `ConwayO7.Encoder.preLex_sat`

source: `ConwayO7/SectionsCompose.lean` ✓

uses: def:enc-prelexinv, prop:enc-deg-section, prop:enc-cn-section, lem:enc-sym1-section, def:phi

Proof. ✓Each of the three component invariants is stable under fresh extension, so each section's triple upgrades to a triple for the full pre-lex invariant; compose the three (Lemma 39) and instantiate at the initial state $(693, \varnothing)$. A separate computation identifies the state built from the specification-level common-neighbour gadgets with the state the concrete generator emits, so the conclusion holds for the clauses of Φ itself. □

3.5 The bridge from strong regularity

Finally, the semantic preconditions above are not ad hoc: they are exactly what strong regularity delivers.

Lemma 57 (Regularity yields the degree invariant)**.** *If G_ω is strongly regular with parameters $(99, 14, 1, 2)$, then the induced valuation a_ω satisfies the degree invariant: for each vertex u the number of true orbit literals at u is the degree of u in G_ω, which is* 14*.*

Lean: `ConwayO7.Encoder.degInv_of_srg`

source: `ConwayO7/Bridge.lean` ✓

uses: def:srg, def:enc-graphoforbits, def:enc-evar, def:enc-deginv

Lemma 58 (The srg identities yield the common-neighbour invariant)**.** *If G_ω is strongly regular with parameters $(99, 14, 1, 2)$, then a_ω satisfies the common-neighbour invariant: for a representative pair r, the count $c_r(a_\omega)$ equals the number of common neighbours of r_1, r_2 plus the adjacency bit, i.e. $\lambda + 1 = 2$ when the pair is an edge and $\mu = 2$ when it is not.*

Lean: `ConwayO7.Encoder.cnInv_of_srg`

source: `ConwayO7/Bridge.lean` ✓

uses: def:srg, def:enc-graphoforbits, def:enc-evar, def:enc-cninv

Proof. ✓The candidate list ranges over all vertices other than r_1, r_2, and a candidate's two orbit literals are true exactly when it is a common neighbour in G_ω; the count is therefore $|N(r_1) \cap N(r_2)|$ plus the adjacency indicator, and both cases of the strong-regularity identity give the value 2 since $\mu - \lambda = 1$. □

Proposition 59 (Strong regularity yields the pre-lex invariant)**.** *If G_ω is strongly regular with parameters $(99, 14, 1, 2)$ and a_ω satisfies the level-1 symmetry condition, then a_ω satisfies the full pre-lex invariant. Combined with Proposition 56, every normalised σ_0-invariant srg would yield*

a valuation satisfying all pre-normalisation clauses of Φ; the minimality clauses are handled in Chapter 4.

Lean: `ConwayO7.Encoder.preLexInv_of_srg`

source: `ConwayO7/Bridge.lean` ✓

uses: lem:enc-deg-bridge, lem:enc-cn-bridge, def:enc-sym1holds, def:enc-prelexinv

Proof. ✓Conjoin Lemmas 57 and 58 with the assumed symmetry condition. □

Chapter 4

Completeness of the normalisation

Conditions (3)–(4) of Definition 34 break symmetry: they demand a normalised neighbourhood for the fixed vertex and lexicographic minimality of the orbit word under 101 relabelling generators. This chapter proves that these conditions lose no graphs: every σ_0-invariant $\mathrm{srg}(99, 14, 1, 2)$ has an isomorphic copy whose orbit word satisfies all of them, and consequently Φ is satisfiable whenever such a graph exists. The argument has three layers: transport of strong regularity along graph isomorphisms, a minimisation over the orbit of the word under the generator action (by well-ordering of $\mathbb{N}$), and the satisfiability of the clauses that render the lexicographic constraints.

4.1 Transport of strong regularity

4.2 The block-relabelling subgroup

Definition 60 (Relabelling generators)**.** Let $N \leq S_{99}$ be generated by: the block transposition exchanging 7-blocks $0, 1$; the transpositions of adjacent blocks $i, i{+}1$ for $2 \leq i \leq 12$; the rotations of each block; and the five multiplier permutations $j \mapsto aj \bmod 7$ $(2 \leq a \leq 6)$ applied simultaneously to all blocks. Each $\rho \in N$ normalises $\langle \sigma_0 \rangle$, hence acts on the orbit space and thus on words $\{0,1\}^{693}$ by $(\rho \cdot \omega)(o) = \omega(\rho^{-1} o)$. The encoding uses 101 fixed generators of this action.

Lean: `ConwayO7.Encoder.lexPerms`, `ConwayO7.Encoder.orbitPerm`

source: `ConwayO7/Encoder.lean` ✓

uses: lem:orbits

Definition 61 (Block relabellings)**.** Write $B_i = \{1 + 7i, \ldots, 7 + 7i\}$ $(0 \leq i \leq 13)$ for the fourteen 7-blocks rotated by σ_0. A permutation $\tau \in S_{14}$ of the block indices induces a permutation of the 99 vertices fixing 0 and sending the j-th vertex of block i to the j-th vertex of block $\tau(i)$: block indices are permuted, positions within blocks are kept.

Lean: `ConwayO7.Encoder.cycleRelabel`

source: `ConwayO7/Sym1Exists.lean` ✓

uses: def:sigma0

Lemma 62 (Relabelling blocks commutes with rotating within blocks)**.** *Every block relabelling commutes with σ_0.*

Lean: `ConwayO7.Encoder.cycleRelabel_comm`, `ConwayO7.Encoder.cycleRelabelVal_comm`

source: `ConwayO7/Sym1Exists.lean` ✓

uses: def:lex-cycle-relabel, def:sigma0

Proof sketch. In the coordinates (block index, position within block) of a nonzero vertex, σ_0 increments the position modulo 7 and fixes the block index, while a block relabelling permutes the block index and fixes the position; the two actions touch disjoint coordinates. Both maps fix the vertex 0. □

4.3 The neighbourhood of the fixed vertex

Computed Fact 63 (Pair orbits at the fixed vertex)**.** For $v \neq 0$ the orbit of the pair $\{0, v\}$ under $\langle\sigma_0\rangle$ depends only on the block of v: it equals the orbit of $\{0, b\}$ where b is the first vertex of v's block. (Checked by computation over all 98 vertices.)

Lean: `Conway07.Encoder.orbit0_cyc`

source: `Conway07/Sym1Exists.lean` ✓

uses: def:sigma0, lem:orbits

Definition 64 (Block bits)**.** For a word $\omega \in \{0,1\}^{693}$ and a block index c, the *block bit* of c is the value of ω on the orbit of the pair $\{0, 1 + 7c\}$, i.e. the adjacency bit of the fixed vertex towards block c.

Lean: `Conway07.Encoder.cycleBit`

source: `Conway07/Sym1Exists.lean` ✓

uses: lem:orbits, fact:lex-orbit-block

Lemma 65 (Adjacency to a block is constant on the block)**.** *In the graph G_ω associated to a word ω, the fixed vertex 0 is adjacent to a vertex v if and only if $v \neq 0$ and the block bit of v's block equals 1. In particular the neighbourhood of 0 is a union of full blocks.*

Lean: `Conway07.Encoder.adj_zero_iff`

source: `Conway07/Sym1Exists.lean` ✓

uses: prop:orbitfun, def:lex-block-bit, fact:lex-orbit-block

Proof sketch. Adjacency in G_ω is the value of ω on the orbit of the pair, and by Fact 63 that orbit depends only on the block of v. □

Computed Fact 66 (Block sizes)**.** Each of the fourteen blocks contains exactly 7 vertices.

Lean: `Conway07.Encoder.cycle_fiber_card`

source: `Conway07/Sym1Exists.lean` ✓

uses: def:sigma0

Lemma 67 (Exactly two neighbour blocks)**.** *If G_ω is an* $\mathrm{srg}(99, 14, 1, 2)$*, then exactly two of the fourteen block bits of ω equal 1.*

Lean: `Conway07.Encoder.two_neighbor_cycles`

source: `Conway07/Sym1Exists.lean` ✓

uses: def:srg, lem:lex-adj-block, fact:lex-block-card

Proof sketch. By regularity the fixed vertex has 14 neighbours; by Lemma 65 its neighbourhood is the disjoint union of the blocks whose bit is 1, each of size 7 (Fact 66). Counting fiberwise, $14 = 7m$ where m is the number of blocks with bit 1, so $m = 2$. □

Lemma 68 (Normalisation of the fixed vertex's neighbourhood)**.** *Every σ_0-invariant* $\mathrm{srg}(99, 14, 1, 2)$ *is isomorphic, via a block relabelling commuting with σ_0, to one in which the neighbourhood of the fixed vertex 0 is exactly the union of blocks 0 and 1: its orbit word has block bit 1 exactly at blocks 0 and 1.*

Lean: *ConwayO7.Encoder.sym1_seed, ConwayO7.Encoder.comap_invariant*

source: *ConwayO7/Sym1Exists.lean* ✓

uses: prop:orbitfun, def:lex-cycle-relabel, lem:lex-relabel-comm, lem:lex-two-blocks, lem:grp-srg-transport

Proof sketch. By Lemma 67 the neighbourhood of 0 is the union of two blocks $d_1 \neq d_2$. A product of two transpositions in S_{14} carries $0, 1$ to d_1, d_2; the induced block relabelling (Definition 61) commutes with σ_0 (Lemma 62), so the pulled-back graph is again σ_0-invariant and hence an orbit-word graph; it is strongly regular by transport (Lemma 23), and its block bits are those of ω permuted so that the two bits equal to 1 land at blocks 0 and 1. □

4.4 The generator action on orbit words

Definition 69 (Vertex action of a generator)**.** Each of the 101 relabelling generators of Definition 60 is given by an explicit map on the vertex set $\{0, \ldots, 98\}$; this vertex map induces the generator's action on pair orbits and hence on words.

Lean: ConwayO7.Encoder.permVertex

source: ConwayO7/Complete.lean ✓

uses: def:relabel

Computed Fact 70 (Compatibility with the orbit structure)**.** For each of the 101 generators ρ and every pair of distinct vertices u, v, the orbit index assigned by ρ to the orbit of $\{u, v\}$ is the orbit of $\{\rho(u), \rho(v)\}$: the action on orbit words is induced by the action on vertices. (Checked by computation over all $101 \cdot \binom{99}{2}$ instances.)

Lean: ConwayO7.Encoder.orbit_compat_of_mem, ConwayO7.Encoder.lexPerms_orbit_compat

source: ConwayO7/Complete.lean ✓

uses: def:relabel, def:lex-vertex-action, lem:orbits

Computed Fact 71 (Injectivity of the vertex maps)**.** The vertex map of each of the 101 generators is injective, hence a permutation of the 99 vertices. (Checked by computation.)

Lean: ConwayO7.Encoder.vertex_inj_of_mem, ConwayO7.Encoder.lexPerms_vertex_inj

source: ConwayO7/Complete.lean ✓

uses: def:relabel, def:lex-vertex-action

Lemma 72 (One generator step preserves strong regularity)**.** *If G_ω is an* $\mathrm{srg}(99, 14, 1, 2)$ *and ρ is one of the* 101 *generators, then $G_{\rho\cdot\omega}$ is again an* $\mathrm{srg}(99, 14, 1, 2)$*.*

Lean: *ConwayO7.Encoder.step_srg*

source: *ConwayO7/Complete.lean* ✓

uses: prop:orbitfun, fact:lex-orbit-compat, fact:lex-vertex-inj, lem:grp-srg-transport

Proof sketch. By Fact 71 the vertex map of ρ is a bijection of the vertex set; by Fact 70 it defines a graph isomorphism $G_{\rho\cdot\omega} \cong G_\omega$. Conclude by Lemma 23. □

Lemma 73 (One generator step preserves the normalisation)**.** *Each of the* 101 *generators permutes the fourteen orbits of pairs $\{0, 1 + 7i\}$ among themselves and preserves the predicate "$i < 2$" on their indices; consequently, if the word ω satisfies the normalisation condition (3) of Definition 34, so does $\rho \cdot \omega$.*

Lean: *ConwayO7.Encoder.step_sym1, ConwayO7.Encoder.lexPerms_sym1*

source: *ConwayO7/Complete.lean* ✓

uses: def:relabel, def:phi, def:lex-vertex-action

Proof sketch. The generators normalise the stabiliser structure at the fixed vertex: each sends a block either to another block wholesale, and blocks $0, 1$ to blocks $0, 1$. The corresponding permutation of the fourteen distinguished orbit indices, together with the invariance of "$i < 2$", is checked by computation for each generator; the preservation of condition (3) follows by rewriting each prescribed bit of $\rho \cdot \omega$ as a prescribed bit of ω. □

Definition 74 (Reachability)**.** For $\omega \in \{0,1\}^{693}$, let $\mathcal{R}(\omega)$ be the smallest set of words containing ω and closed under the action of each of the 101 generators: the orbit of ω under the monoid they generate.

Lean: `ConwayO7.Encoder.Reach`

source: `ConwayO7/Complete.lean` ✓

uses: def:relabel

Lemma 75 (Reachability preserves the invariants)**.** *If G_ω is an* $\mathrm{srg}(99, 14, 1, 2)$*, then so is $G_{\omega'}$ for every $\omega' \in \mathcal{R}(\omega)$; and if ω satisfies the normalisation condition, so does every $\omega' \in \mathcal{R}(\omega)$.*

Lean: `ConwayO7.Encoder.reach_srg, ConwayO7.Encoder.reach_sym1`

source: `ConwayO7/Complete.lean` ✓

uses: def:lex-reach, lem:lex-step-srg, lem:lex-step-sym1

Proof sketch. Induction along the chain of generator steps, using Lemmas 72 and 73 at each step. □

4.5 Word values and minimisation

Definition 76 (Big-endian value)**.** The *value* of a word $\omega \in \{0,1\}^{693}$ is $\mathrm{val}(\omega) = \sum_{i<693} \omega(i)\, 2^{692-i}$, the word read as a big-endian binary number.

Lean: `ConwayO7.Encoder.wordVal`

source: `ConwayO7/LexVal.lean` ✓

uses: lem:orbits

Lemma 77 (Value order refines lexicographic order)**.** *Let $u, w \in \{0,1\}^{693}$ with $\mathrm{val}(u) \le \mathrm{val}(w)$. Then for every position $i < 693$: if u and w agree at all positions $j < i$ and $u(i) = 1$, then $w(i) = 1$. Equivalently, $u \preceq w$ in the lexicographic order.*

Lean: `ConwayO7.Encoder.lex_of_val_le`

source: `ConwayO7/LexVal.lean` ✓

uses: def:lex-word-val

Proof sketch. Suppose $u(i) = 1$ but $w(i) = 0$ with equal prefixes below i. Split both values at position i: the prefix contributions coincide, the bit of u at i contributes 2^{692-i}, and the tail of w is at most $\sum_{t<692-i} 2^t = 2^{692-i} - 1$. Hence $\mathrm{val}(u) > \mathrm{val}(w)$, a contradiction. □

Lemma 78 (Existence of a reachable minimum)**.** *For every ω there is $\omega_{\min} \in \mathcal{R}(\omega)$ with $\mathrm{val}(\omega_{\min}) \le \mathrm{val}(\omega')$ for all $\omega' \in \mathcal{R}(\omega)$.*

Lean: `ConwayO7.Encoder.exists_min_reach`

source: `ConwayO7/Complete.lean` ✓

uses: def:lex-reach, def:lex-word-val

Proof sketch. The set of values of reachable words is a nonempty set of natural numbers; by well-ordering it has a least element, attained by some reachable word. □

4.6 The comparison chains

Definition 79 (Lexicographic comparison of literal words)**.** Fix two sequences X, Y of 693 literals over the orbit variables — for the encoding, X lists the orbit variables in position order and Y lists them re-indexed by a generator's orbit action. A valuation a satisfies $X \preceq Y$ if for every $i < 693$: whenever the literal values of X and Y agree at all positions $j < i$ and the i-th literal of X is true, the i-th literal of Y is true. This is the lexicographic comparison of the two literal-value words under a.

Lean: `ConwayO7.Encoder.LexLeq, ConwayO7.Encoder.lexXA, ConwayO7.Encoder.lexYA`

source: `ConwayO7/LexSpec.lean` ✓

uses: def:phi, def:relabel

Definition 80 (The lex-leader invariant)**.** A valuation of the orbit variables satisfies the *lex-leader invariant* if for each of the 101 generators ρ it satisfies $X \preceq Y_\rho$, where Y_ρ is X re-indexed by ρ's orbit action. On words this says precisely $\omega \preceq \rho \cdot \omega$ for every generator ρ, i.e. condition (4) of Definition 34.

Lean: `ConwayO7.Encoder.LexInv`

source: `ConwayO7/LexSpec.lean` ✓

uses: def:lex-leq, def:relabel, def:phi

Lemma 81 (Satisfiability of one comparison chain)**.** *The clauses rendering one comparison $X \preceq Y$ use fresh auxiliary registers. Every valuation a that satisfies all previously emitted clauses and the comparison $X \preceq Y$ (on literal values) extends, by assigning only the fresh registers, to a valuation satisfying all clauses of the chain as well.*

Lean: `ConwayO7.Encoder.specAddLexLeq_sat, ConwayO7.Encoder.specAddLexLeq`

source: `ConwayO7/LexSpec.lean` ✓

uses: def:lex-leq, def:phi

Proof sketch. The chain introduces, at each position where the two literals differ structurally, one register meaning "all earlier positions agree", defined by clauses from its predecessor and the current pair, together with the implication (i-th literal of X true $\Rightarrow$ i-th literal of Y true) guarded by the previous register. Assign each register its intended truth value — the running equality flag of the literal-value words. Every defining clause then holds by construction, and every guarded implication holds because the comparison $X \preceq Y$ supplies exactly the implication at each position whose strict prefix agrees. Induction over the positions of the chain. □

Proposition 82 (The comparison section)**.** *The full minimality section of the encoding — the* 101 *chained comparisons of condition (4) — preserves satisfiability: any valuation of the orbit block satisfying the lex-leader invariant extends over the auxiliary registers of the section to satisfy every clause the section emits.*

Lean: `ConwayO7.Encoder.lex_section_trip, ConwayO7.Encoder.specAddLexLeq_trip, ConwayO7.Encoder.specLexAgainst_trip`

source: `ConwayO7/LexTrip.lean` ✓

uses: def:lex-inv, lem:lex-chain-sat

Proof sketch. Each of the 101 chains is handled by Lemma 81, whose hypothesis is one conjunct of the lex-leader invariant; the extensions compose because each chain touches only its own fresh registers, leaving the orbit block — and hence the invariant — unchanged. □

4.7 Assembly

Theorem 83 (Completeness of minimality)**.** *Every σ_0-invariant* $\mathrm{srg}(99, 14, 1, 2)$ *admits an isomorphic copy whose orbit word ω satisfies conditions (3)–(4) of Definition 34: the normalisation of Lemma 68 and the lex-leader invariant $\omega \preceq \rho \cdot \omega$ for each of the* 101 *generators ρ.*

Lean: `ConwayO7.Encoder.lexLeaderComplete, ConwayO7.Encoder.LexLeaderComplete`

source: `ConwayO7/Complete.lean` ✓

uses: lem:sym1, def:lex-reach, lem:lex-reach-inv, lem:lex-min-reach, lem:lex-value-order, def:lex-inv

Proof sketch. Normalise first: by Lemma 68 pass to a word ω_0 that is strongly regular and normalised. Among the words reachable from ω_0 (Definition 74), choose one of minimal value, $\omega_{\min}$ (Lemma 78); by Lemma 75 it is still strongly regular and normalised. For each generator ρ the word $\rho \cdot \omega_{\min}$ is again reachable, so $\mathrm{val}(\omega_{\min}) \le \mathrm{val}(\rho \cdot \omega_{\min})$, and by Lemma 77 value order refines lexicographic order: $\omega_{\min} \preceq \rho \cdot \omega_{\min}$. Translating the word-level comparison into the literal-level comparison of Definition 79 gives the lex-leader invariant. □

Definition 84 (The master invariant)**.** The *master invariant* of a valuation of the orbit block is the conjunction of the semantic conditions of all four sections of the encoding: the degree counts (1), the common-neighbour counts (2), the normalisation (3), and the lex-leader invariant (4).

Lean: `ConwayO7.Encoder.FullInv`

source: `ConwayO7/Assembly.lean` ✓

uses: def:phi, prop:counters, def:lex-inv

Proposition 85 (Generator satisfiability)**.** *Every valuation of the orbit block satisfying the master invariant extends, over the auxiliary variables, to a valuation satisfying every clause of the generated formula Φ.*

Lean: `ConwayO7.Encoder.mkO7St_sat, ConwayO7.Encoder.fullSpec_trip`

source: `ConwayO7/Assembly.lean` ✓

uses: def:lex-full-inv, prop:counters, prop:lex-section-trip, def:phi

Proof sketch. The four sections are emitted in sequence, each consuming one conjunct of the master invariant and preserving the rest (each section's auxiliary variables are fresh, so the other conjuncts, which depend only on the orbit block, are untouched). The counting sections extend by Proposition 56, the comparison section by Proposition 82; composing the four extension steps yields a satisfying valuation of the whole formula. That the abstractly described pipeline emits exactly the clauses of Φ is checked by computation. □

Lemma 86 (Faithfulness from completeness of minimality)**.** *Assume completeness of minimality (Theorem 83). If a σ_0-invariant* $\mathrm{srg}(99, 14, 1, 2)$ *exists, then the distributed formula is satisfiable.*

Lean: `ConwayO7.Encoder.encodes_of_lexLeader`

source: `ConwayO7/Assembly.lean` ✓

uses: prop:orbitfun, lem:grp-srg-transport, def:lex-full-inv, prop:lex-generator-sat, fact:bytes

Proof sketch. An invariant graph descends to an orbit word ω (Proposition 31) whose associated graph is strongly regular by transport (Lemma 23). Completeness of minimality replaces ω by a companion word satisfying strong regularity, the normalisation, and the lex-leader invariant; the arithmetic conditions (1)–(2) hold for the orbit word of any strongly regular graph, so the companion satisfies the master invariant. Proposition 85 extends it to a satisfying valuation of the generated formula, which equals the distributed file by the byte identity (Fact 37). □

Theorem 87 (Completeness of the encoding)**.** *If a σ_0-invariant* $\mathrm{srg}(99, 14, 1, 2)$ *exists, then* Φ *is satisfiable.*

Lean: `ConwayO7.Encoder.encodes_o7`

source: `ConwayO7/Complete.lean` ✓

uses: thm:lex-complete, lem:lex-encodes

Proof sketch. Combine Theorem 83 with Lemma 86. □

Chapter 5

Propositional formulas and the covering partition

This chapter develops, from first principles, the small amount of propositional logic that the certificate rests on, and then the combinatorial heart of the matter: the 98,536 cells of the distributed certificate partition the whole space of valuations. No background in logic is assumed; every notion is a finite combinatorial object over the two-element set $\{0,1\}$. (In Lean, $\{0,1\}$ is the type `Bool` with elements `false`, `true`; we write 0 and 1.)

5.1 Propositional formulas from first principles

Definition 88 (Variables and literals)**.** Fix a countable index set V of *variables*; concretely $V = \{0,1,2,\dots\}$. A *literal* is a pair $(x,\varepsilon) \in V \times \{0,1\}$, written x when $\varepsilon = 1$ and $\neg x$ when $\varepsilon = 0$: an assertion that the variable x takes the value ε.

Lean: `Std.Sat.Literal`

source: `mathlib / Lean core` ✓

Definition 89 (Clauses and formulas)**.** A *clause* is a finite list of literals, read *disjunctively* ("at least one of these assertions holds"). A *formula in conjunctive normal form* (a *CNF formula*) is a finite family $F = (C_1,\dots,C_m)$ of clauses, read *conjunctively* ("every clause holds").

Lean: `Std.Sat.CNF, Std.Sat.CNF.Clause`

source: `mathlib / Lean core` ✓

uses: def:cnf-literal

Definition 90 (Valuations and satisfaction)**.** A *valuation* is a function $a\colon V \to \{0,1\}$, assigning a value to every variable. A valuation a *satisfies*

- the literal (x,ε) iff $a(x) = \varepsilon$;
- a clause C iff it satisfies *at least one* literal of C;
- a CNF formula F iff it satisfies *every* clause of F.

In Lean these are the evaluation maps sending a valuation and a clause/formula to $\{0,1\}$; satisfaction means evaluating to 1.

Lean: `Std.Sat.CNF.eval, Std.Sat.CNF.Clause.eval`

source: `mathlib / Lean core` ✓

uses: def:cnf-formula

Definition 91 (Satisfiability)**.** A CNF formula F is *satisfiable* if some valuation satisfies it, and *unsatisfiable* if no valuation does. Unsatisfiability of a suitable formula is the finitary statement to which the non-existence theorem of this project is reduced.

Lean: `Std.Sat.CNF.Sat, Std.Sat.CNF.Unsat`

source: `mathlib / Lean core` ✓

uses: def:cnf-valuation

Definition 92 (Cubes: partial valuations)**.** A *cube* is a finite list of literals read *conjunctively* ("all of these assertions hold") — dually to a clause. A valuation satisfies a cube c iff it satisfies every literal of c; thus a cube whose variables are pairwise distinct is the same thing as a *partial valuation* with finite domain, and its satisfaction set is the set of all total valuations extending it.

Lean: `ConwayO7.Cube, ConwayO7.Cube.eval`

source: `ConwayO7/Semantics.lean` ✓

uses: def:cnf-literal, def:cnf-valuation

Definition 93 (Adjoining a cube to a formula)**.** For a CNF formula F and a cube c, the formula $F \wedge c$ is obtained by adjoining to F one singleton clause $\{\ell\}$ per literal ℓ of c. This is the formula whose refutation certifies that F has no satisfying valuation *inside the cell described by* c.

Lean: `ConwayO7.withCube`

source: `ConwayO7/Semantics.lean` ✓

uses: def:cnf-formula, def:cnf-cube

Lemma 94 (Semantics of $F \wedge c$)**.** *A valuation satisfies $F \wedge c$ if and only if it satisfies both F and the cube c.*

Lean: `ConwayO7.eval_withCube`

source: `ConwayO7/Semantics.lean` ✓

uses: def:cnf-withcube, def:cnf-valuation

Proof sketch. Induction on the list c: a singleton clause $\{\ell\}$ is satisfied exactly when the literal ℓ is, and adjoining a clause intersects the satisfaction sets. □

Theorem 95 (Composition of refutations)**.** *Let F be a CNF formula and let $c_1, \ldots, c_N$ be cubes whose satisfaction sets* cover *the space of valuations (every valuation satisfies some c_i). If $F \wedge c_i$ is unsatisfiable for every i, then F is unsatisfiable.*

Lean: `ConwayO7.unsat_of_cubes`

source: `ConwayO7/Semantics.lean` ✓

uses: def:cnf-unsat, def:cnf-withcube

Proof sketch. *uses: lem:cnf-withcube-eval* Any valuation satisfying F satisfies some c_i, hence by Lemma 94 it satisfies $F \wedge c_i$, contradicting the i-th refutation. □

Definition 96 (The audited text convention)**.** The distributed files render literals as signed integers: the literal $(v, 1)$ (with v counted from 0) is written as the integer $v+1$, and $(v, 0)$ as $-(v+1)$. A clause is a line of literals terminated by 0; a cube is a line of literals with no terminator. The parsing maps from text to formulas and cubes, and the serialisation maps in the other direction, are fixed once in a deliberately small Lean file; on the artifact files they are mutually inverse byte for byte, which is what gives the byte-identity facts (such as Fact 37) their meaning.

Lean: `ConwayO7.Dimacs.litOfInt, ConwayO7.Dimacs.parseDimacs, ConwayO7.Dimacs.serializeDimacs, ConwayO7.Dimacs.pars`
`ConwayO7.Dimacs.serializeCubes`

source: `ConwayO7/Dimacs.lean` ✓

uses: def:cnf-formula, def:cnf-cube

5.2 Sign-prefix cells and Kraft mass

The cubes of the certificate are not arbitrary: they all constrain an initial segment of one fixed sequence of variables. Coverage therefore reduces to a purely combinatorial statement about finite words over $\{0,1\}$.

Definition 97 (Sign prefixes and cells)**.** Fix a *branching order*: a sequence of D distinct variables, in the certificate the sequence $x_{15}, x_{16}, \ldots, x_{214}$ (so $D = 200$). To a word $s \in \{0,1\}^{\le D}$ associate the cube asserting $x_{14+j} = s_j$ for $1 \le j \le |s|$; its satisfaction set, the *cell* of s, consists of all valuations extending the partial valuation s along the branching order. A family S of words is *prefix-free* if no member is a prefix of another (in particular the members are pairwise distinct).

Lean: `ConwayO7.cubeOfSigns, ConwayO7.PrefixFree`

source: `ConwayO7/Coverage.lean` ✓

uses: def:cnf-cube

Lemma 98 (Cells are prefix sets)**.** *Write $w(a) \in \{0,1\}^D$ for the* branch word *of a valuation a: the values of a along the branching order. A valuation a lies in the cell of a word s (with $|s| \le D$) if and only if s is a prefix of $w(a)$. Consequently two cells are either nested or disjoint, and the cells of a prefix-free family are pairwise disjoint.*

Lean: `ConwayO7.cubeOfSigns_eval_iff`

source: `ConwayO7/Coverage.lean` ✓

uses: def:prefix, def:cnf-valuation

Proof sketch. Induction on s along the branching order. □

Definition 99 (Kraft weight)**.** The *Kraft weight* at depth D of a family S of words of length $\le D$ is the integer $W_D(S) = \sum_{s \in S} 2^{\,D-|s|}$, i.e. 2^D times the usual Kraft mass $\sum_s 2^{-|s|}$. The cell of s meets exactly $2^{\,D-|s|}$ of the 2^D branch words, so $W_D(S)$ is the total number of branch words covered, counted with multiplicity.

Lean: `ConwayO7.kraftWeight`

source: `ConwayO7/Coverage.lean` ✓

uses: def:prefix

Lemma 100 (Splitting at the root)**.** *For $b \in \{0,1\}$ let $S_b = \{t : b\,t \in S\}$ be the family of tails of the members of S beginning with b. If the empty word is not in S, then $W_{D+1}(S) = W_D(S_0) + W_D(S_1)$; moreover each S_b inherits prefix-freeness and the length bound from S.*

Lean: `ConwayO7.kraftWeight_split, ConwayO7.branchTails, ConwayO7.prefixFree_branchTails`

source: `ConwayO7/Coverage.lean` ✓

uses: def:cov-kraft, def:prefix

Proof sketch. Each $s \in S$ starts with exactly one bit b and contributes $2^{(D+1)-|s|} = 2^{D-|s_{\ge 2}|}$ to the b-branch. Prefix relations between words with the same first bit correspond exactly to prefix relations between their tails. □

Lemma 101 (Kraft's inequality)**.** *A prefix-free family $S \subseteq \{0,1\}^{\le D}$ satisfies $W_D(S) \le 2^D$.*

Lean: `ConwayO7.kraftWeight_le`

source: `ConwayO7/Coverage.lean` ✓

uses: def:cov-kraft, def:prefix

Proof sketch. *uses: lem:cov-kraft-split* Induction on D. If the empty word belongs to S then prefix-freeness forces $S = \{\epsilon\}$, of weight exactly 2^D; otherwise split at the root by Lemma 100 and apply the inductive bound to each branch. □

Lemma 102 (Kraft equality and maximality)**.** *A prefix-free family $S \subseteq \{0,1\}^{\le D}$ with full Kraft weight $W_D(S) = 2^D$ is a* maximal antichain *of the binary tree: every $w \in \{0,1\}^D$ has a (necessarily unique) member of S as a prefix. Hence the corresponding cells* partition *the valuation space.*

Lean: `ConwayO7.covers_of_prefixFree_kraft`

source: `ConwayO7/Coverage.lean` ✓

uses: def:prefix, def:cov-kraft

Proof sketch. *uses: lem:cov-kraft-split, lem:cov-kraft-ineq* Induction on D, following the given word w bit by bit. If $\epsilon \in S$ we are done. Otherwise split at the root: by Lemma 100 the two branch weights sum to $2^{D+1} = 2^D + 2^D$, and by Kraft's inequality neither exceeds 2^D, so *both* branches again have full weight; recurse into the branch selected by the first bit of w. Uniqueness and disjointness follow from prefix-freeness via Lemma 98. □

Lemma 103 (Coverage of the valuation space)**.** *Let S be a prefix-free family of words of length $\le D$ with full Kraft weight 2^D. Then every valuation lies in the cell of some (unique) member of S.*

Lean: `ConwayO7.cube_cover_of_signs`

source: `ConwayO7/Coverage.lean` ✓

uses: def:prefix, def:cnf-valuation

Proof sketch. *uses: lem:kraft, lem:cov-cell-prefix* Apply Lemma 102 to the branch word $w(a)$ of the given valuation a and translate back through Lemma 98. □

5.3 An executable coverage check

The concrete family has 98,536 members, so verifying prefix-freeness literally — all $\sim 5 \cdot 10^9$ pairs — is out of reach. Instead the family is sorted lexicographically and only *adjacent* pairs are compared; the block structure of the lexicographic order makes this sufficient.

Definition 104 (Lexicographic order on words)**.** Order $\{0,1\}^{\le D}$ by $s \le t$ iff at the first position where they differ (reading left to right) s carries 0 and t carries 1, or s is a prefix of t. This relation is total, transitive and antisymmetric: a linear order on words.

Lean: `ConwayO7.lexLe, ConwayO7.lexLe_total, ConwayO7.lexLe_trans, ConwayO7.lexLe_antisymm`

source: `ConwayO7/CoverageCheck.lean` ✓

Lemma 105 (Block property of the lexicographic order)**.** *Extensions of a fixed word s form an* interval*: if s is a prefix of t and $s \le u \le t$ in the lexicographic order, then s is a prefix of u. (In particular $s \le t$ whenever s is a prefix of t.)*

Lean: `ConwayO7.prefix_of_lexLe_of_lexLe, ConwayO7.lexLe_of_prefix`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:cov-lex

Proof sketch. Simultaneous induction on the three words: matching leading bits are stripped, and a strict inequality at the head contradicts one of the two order hypotheses. □

Definition 106 (The adjacent-pairs test)**.** For a finite sequence of words, the *adjacent-pairs test* checks, in one pass, that no term is a prefix of its immediate successor.

Lean: `ConwayO7.adjacentOk`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:prefix

Lemma 107 (Soundness of the adjacent-pairs test)**.** *If a sequence of words is sorted for the lexicographic order and passes the adjacent-pairs test, then the underlying family is prefix-free.*

Lean: `ConwayO7.pairwise_of_sorted_adjacent`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:cov-adjacent, def:cov-lex, def:prefix

Proof sketch. *uses: lem:cov-lex-block* Suppose s precedes u in the sorted sequence and s is a prefix of u, with immediate successor t of s. Then $s \le t \le u$, so by the block property (Lemma 105) s is already a prefix of its neighbour t — excluded by the test. The reverse prefix relation would force $s = u$ by antisymmetry. □

Definition 108 (The coverage check)**.** Given a depth D and a family S of words, the *coverage check* is the finite computation verifying: (i) every member of S has length $\le D$; (ii) the family, sorted lexicographically (by merge sort), passes the adjacent-pairs test; (iii) the Kraft weight satisfies $W_D(S) = 2^D$ exactly, in integer arithmetic.

Lean: `ConwayO7.checkCover`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:cov-adjacent, def:cov-lex, def:cov-kraft

Lemma 109 (Soundness of the coverage check)**.** *If the coverage check succeeds on (D, S), then every member of S has length $\le D$, the family S is prefix-free, and $W_D(S) = 2^D$ — the three hypotheses of Lemma 103.*

Lean: `ConwayO7.cover_of_checkCover`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:cov-checkcover, def:prefix, def:cov-kraft

Proof sketch. *uses: lem:cov-adjacent-sound* The sorted sequence is a permutation of S and is pairwise ordered (both facts are standard properties of merge sort, available in the library), so Lemma 107 yields pairwise prefix-freeness, which transfers across the permutation; the length and weight conditions are checked directly. □

Theorem 110 (Refutation by cells)**.** *Let F be a CNF formula, fix a branching order of length D, and let $S \subseteq \{0,1\}^{\le D}$ be prefix-free with full Kraft weight $W_D(S) = 2^D$. If for every $s \in S$ the formula $F \wedge (\mathit{cell}\ s)$ is unsatisfiable, then F is unsatisfiable.*

Lean: `ConwayO7.unsat_of_prefix_cover`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: def:prefix, def:cov-kraft, def:cnf-withcube, def:cnf-unsat

Proof sketch. *uses: thm:cnf-unsat-of-cubes, lem:cov-valuation-cover* By Lemma 103 the cells of S cover the valuation space; conclude by the composition theorem (Theorem 95). □

Corollary 111 (Refutation by checked cells)**.** *If the coverage check succeeds on (D, S) and $F \wedge (\mathit{cell}\ s)$ is unsatisfiable for every $s \in S$, then F is unsatisfiable. The entire coverage side of the certificate thus reduces to one finite computation on the concrete data.*

Lean: `ConwayO7.unsat_of_checked_cover`

source: `ConwayO7/CoverageCheck.lean` ✓

uses: thm:coverage, def:cov-checkcover

Proof sketch. *uses: lem:cov-checkcover-sound* Combine Lemma 109 with Theorem 110. □

5.4 The distributed family

Definition 112 (The certified tiling data)**.** The consolidated cell list of the artifact (one cube per certificate, in manifest order) is embedded byte for byte into the Lean development and parsed by the maps of Definition 96, yielding the list of *tiling cubes*; the *branching order* is the sequence $x_{15}, \ldots, x_{214}$ of 200 variables; the *sign words* are the words in $\{0,1\}^{\leq 200}$ obtained by reading off the signs of each cube's literals. No processing external to Lean intervenes between the hash-pinned file and these terms.

Lean: `ConwayO7.cubesAllTxt, ConwayO7.tilingCubes, ConwayO7.tilingOrder, ConwayO7.tilingSigns`

source: `ConwayO7/Data/FullTiling.lean` ✓

uses: def:cnf-cube, def:cnf-dimacs

Computed Fact 113 (Faithfulness of the sign-word model)**.** Rebuilding each cube from its sign word along the branching order reproduces the parsed cube list exactly: every distributed cube really is a sign pattern on an initial segment of consecutive variables starting at x_{15}. (If the file contained any cube of another shape, this computation would fail.)

Lean: `ConwayO7.tilingSigns_eq`

source: `ConwayO7/Data/FullTiling.lean` ✓

uses: def:cov-tiling, def:prefix

Computed Fact 114 (Cardinality)**.** The distributed family consists of exactly 98,536 sign words, of lengths between 12 and 200.

Lean: `ConwayO7.tiling_count`

source: `ConwayO7/Data/FullTiling.lean` ✓

uses: def:cov-tiling

Computed Fact 115 (The distributed family tiles the valuation space)**.** The coverage check of Definition 108 succeeds, at depth 200, on the 98,536 sign words: the family is prefix-free with full Kraft weight 2^{200}, so by Lemma 102 its cells partition the valuation space — a perfect tiling. This is established by executing the verified check, inside Lean's compiled evaluator, on the embedded data.

Lean: `ConwayO7.o7Tiling_checkCover`

source: `ConwayO7/Data/FullTiling.lean` ✓

uses: def:cov-tiling, def:cov-checkcover, def:prefix

Chapter 6

Certificates

Definition 116 (Clausal derivations)**.** Let F be a formula in conjunctive normal form, i.e. a finite set of *clauses*, each a disjunction of literals. A clause C is *entailed by reverse unit propagation* (RUP) from a set G of clauses if, starting from the assumption that every literal of C is false and repeatedly assigning any literal that is forced (because it is the last unfalsified literal of some clause of G), one reaches a clause of G all of whose literals are falsified; this syntactic check implies the semantic entailment $G \models C$, since any valuation satisfying G but falsifying C would survive every propagation step. A *clausal derivation* from F is a finite sequence of clauses, each entailed by reverse unit propagation from F together with the preceding clauses (with clause deletions permitted along the way), ending in the empty clause — the clause with no literals, satisfied by no valuation. Since RUP steps preserve entailment, a clausal derivation witnesses that F is unsatisfiable. The *LRAT format* additionally annotates each step with the indices of the clauses used, so that a derivation can be checked in time linear in its length; Lean's standard library formalises LRAT derivations as sequences of addition and deletion actions.

Lean: `Std.Tactic.BVDecide.LRAT.IntAction`, `Std.Tactic.BVDecide.LRAT.parseLRATProof`

source: `mathlib / Lean core` ✓

Theorem 117 (The verified checker)**.** *Lean's standard library provides an executable checker taking an LRAT derivation and a CNF, and a soundness theorem proved in Lean: whenever the checker accepts, the formula is unsatisfiable. Consequently a run of the checker inside Lean's kernel converts a concrete derivation into a genuine Lean proof of unsatisfiability, with no unverified translation step.*

Lean: *`Std.Tactic.BVDecide.LRAT.check`, `Std.Tactic.BVDecide.LRAT.check_sound`*

source: *`mathlib / Lean core`* ✓

uses: def:lrat

Computed Fact 118 (In-kernel sample replay)**.** Cell number 4092 of the distributed family (historically cube 4093 of the first-round split) is unsatisfiable together with Φ: the distributed derivation for this cell, whose compressed file is pinned by its SHA-256 hash, is replayed through the verified checker of Theorem 117 against the formula $\Phi \wedge$ (cell) built from the byte-identical embedded data by the same cube-adjunction used throughout the pipeline. This is the calibration sample: a sign convention, off-by-one, or clause-order mismatch anywhere between the Lean development and the distributed artifact would make this check fail.

Lean: `Conway07.cube4093_unsat`

source: `Conway07/Data/Sample.lean` ✓

uses: def:lrat, crt-checker, fact:bytes, fact:tiling

Computed Fact 119 (The external hypothesis)**.** For each of the 98,536 cells there is a clausal derivation refuting $\Phi \wedge (\text{cell } s)$. Each derivation was checked by cake_lpr, an LRAT checker formally verified (in HOL4) down to the machine code that executes; verification was performed on the distributed files, whose hashes are pinned. This is the unique hypothesis of the development not discharged inside Lean; one of the 98,536 instances is additionally replayed inside Lean's kernel (Fact 118), eliminating even the translation gap for that sample.

uses: def:lrat, crt-sample, fact:bytes, fact:tiling

Theorem 120 (The certificate schema)**.** *Let F be a CNF, and let S be a prefix-free family of sign words of full Kraft mass over a fixed variable order (Definition 97, Lemma 102). If $F \wedge (\text{cell } s)$ is unsatisfiable for every $s \in S$, and F encodes the σ_0-invariant strongly regular graphs (Definition 4), then no strongly regular graph with parameters $(99, 14, 1, 2)$ has automorphism group of order divisible by 7. This composes the refutation-by-cells theorem (Theorem 110) with the conditional main theorem (Theorem 5); a variant takes the prefix-freeness and Kraft hypotheses in the executable form of the cover checker.*

Lean: `ConwayO7.no_aut_seven_of_certificates, ConwayO7.no_aut_seven_of_checked_certificates`

source: `ConwayO7/Pipeline.lean` ✓

uses: st-conditional, st-encodes, thm:coverage, def:prefix, lem:kraft

Theorem 121 (The schema on the distributed family)**.** *Instantiate Theorem 120 with $F = \Phi$ and S the distributed family of 98,536 words: if $\Phi \wedge (\text{cell } s)$ is unsatisfiable for every s in the family, and Φ encodes the σ_0-invariant strongly regular graphs, then the conclusion of the main theorem holds. The combinatorial hypotheses on S are discharged by the executed cover check (Fact 115); the remaining hypotheses are exactly the per-cell unsatisfiability facts and encoding completeness.*

Lean: `ConwayO7.no_aut_seven_of_tiling_certificates`

source: `ConwayO7/Data/FullTiling.lean` ✓

uses: crt-schema, fact:tiling, def:phi

Assembly of Theorem 3. uses: crt-tiling, thm:encoding-complete, fact:certificates Apply Theorem 121. Its per-cell hypothesis is Fact 119: every $\Phi \wedge (\text{cell } s)$ for s in the distributed family is unsatisfiable. Its encoding hypothesis is Theorem 87: Φ encodes the σ_0-invariant $\mathrm{srg}(99, 14, 1, 2)$s. Hence no $\mathrm{srg}(99, 14, 1, 2)$, on any finite vertex set, has automorphism group of order divisible by 7. □

Trusted base

The formal proof rests on: the Lean 4 kernel and mathlib; nineteen named computations executed by Lean's compiled evaluator (each a finite check over the embedded data, including Facts 37 and 115); the HOL4-verified checker for Fact 119; and the identity of the two files (`o7.cnf` and the cell list) shared between the Lean development and the certificate checking — by construction when both halves are run from one copy of the repository.